Objetivos da aula:

- Compreender o papel dos padrões comportamentais no design de sistemas orientados a objetos.
- Identificar problemas relacionados à comunicação e interação entre objetos em aplicações de software.
- Conhecer padrões comportamentais comuns e compreender suas aplicações práticas.
- Aplicar esses padrões em exemplos práticos utilizando TypeScript.
- Compreender como esses padrões aparecem em arquiteturas modernas, como aplicações baseadas em Node.js e React.

Índice:

i. Introdução aos Padrões Comportamentais
ii. Observer Pattern
iii. Strategy Pattern
iv. Command Pattern
v. State Pattern
vi. Chain of Responsability Pattern

i. Introdução aos Padrões Comportamentais

1.1. Introdução

No desenvolvimento de software orientado a objetos, não basta apenas organizar corretamente a criação de objetos ou a estrutura das classes. Também é necessário definir **como os objetos interagem entre si** dentro do sistema.

Em aplicações maiores, diferentes partes do sistema precisam trocar informações, coordenar ações ou reagir a mudanças de estado. Quando essa comunicação não é bem estruturada, o código pode se tornar:

- difícil de entender
- fortemente acoplado
- difícil de modificar
- propenso a erros

Os **padrões de projeto comportamentais** surgem justamente para organizar essas interações.

Esses padrões descrevem formas padronizadas de estruturar **a comunicação entre objetos**, definindo responsabilidades claras e reduzindo dependências desnecessárias.

Em outras palavras, os padrões comportamentais ajudam a responder perguntas como:

- Como objetos devem se comunicar entre si?
- Como distribuir responsabilidades dentro de um sistema?
- Como permitir diferentes comportamentos sem modificar o código principal?

Ao aplicar esses padrões, o sistema tende a se tornar **mais flexível, modular e fácil de manter**.

1.2. O que são Padrões Comportamentais

Os padrões comportamentais são padrões de projeto que tratam da **forma como objetos interagem e colaboram entre si**.

Enquanto:

- padrões criacionais tratam da criação de objetos
- padrões estruturais tratam da organização das classes

os padrões comportamentais se concentram em **como as responsabilidades são distribuídas e como os objetos se comunicam**.

Esses padrões procuram reduzir o acoplamento entre diferentes partes do sistema e tornar a lógica de interação mais clara e reutilizável.

Entre os principais objetivos desses padrões estão:

- organizar a comunicação entre objetos
- reduzir dependências diretas entre classes
- tornar o comportamento do sistema mais flexível
- facilitar a extensão e manutenção do código

1.3. Exemplos de Padrões Comportamentais

A Gang of Four definiu onze padrões estruturais e alguns dos mais conhecidos são:

1. Chain of Responsibility: passar requisição por uma cadeia de objetos
2. Command: encapsular comandos em objetos
3. Observer: notificar objetos sobre mudanças
4. State: alterar comportamento conforme estado
5. Strategy: trocar algoritmos dinamicamente

Esses padrões aparecem com frequência em:

- frameworks modernos
- bibliotecas de interface
- sistemas distribuídos
- arquiteturas orientadas a eventos

Por exemplo, o padrão Observer é amplamente utilizado em frameworks de interface para reagir a eventos do usuário, enquanto o padrão Strategy é comum em sistemas que precisam alternar diferentes formas de processamento.

ii. Observer Pattern

2.1. Introdução

Em muitos sistemas de software, determinados objetos precisam ser informados quando ocorre alguma mudança em outro objeto.

Esse tipo de situação aparece com frequência em diferentes contextos, como por exemplo:

- atualização de interfaces gráficas
- sistemas de notificações
- aplicações orientadas a eventos
- comunicação entre componentes de uma aplicação

Considere um sistema de vendas online. Quando um pedido é realizado, diversos componentes podem precisar reagir a esse evento:

- o sistema de estoque precisa atualizar a quantidade disponível
- o sistema de pagamento precisa processar a cobrança
- o sistema de notificações pode enviar um e-mail ao cliente

Se cada um desses componentes depender diretamente do objeto que gerou o evento, o sistema pode se tornar fortemente acoplado.

Para resolver esse tipo de problema, podemos utilizar o Observer Pattern.

2.2. O que é o Observer Pattern

O Observer Pattern é um padrão comportamental que define uma dependência do tipo **um-para-muitos** entre objetos.

Quando um objeto muda de estado, todos os objetos que dependem dele são automaticamente notificados.

Esse padrão envolve dois tipos principais de participantes:

- **Subject (Observável)** – objeto que possui o estado observado
- **Observers (Observadores)** – objetos que são notificados quando ocorre uma mudança

Dessa forma, o objeto principal não precisa conhecer os detalhes de cada observador. Ele apenas envia notificações para todos os objetos registrados.

Isso permite que novos observadores sejam adicionados ao sistema **sem modificar o objeto principal**.

2.3. Exemplo em TypeScript

Considere um sistema simples de monitoramento de temperatura.

Primeiro definimos a interface dos observadores.

```
interface Observer {
  update(temperatura: number): void;
}
```

Agora criamos a classe responsável por armazenar e notificar os observadores.

```
class SensorTemperatura {
  private observers: Observer[] = [];
```

```
  private temperatura: number = 0;

  adicionarObserver(observer: Observer) {
    this.observers.push(observer);
  }

  removerObserver(observer: Observer) {
    this.observers = this.observers.filter((o) => o !== observer);
  }

  setTemperatura(valor: number) {
    this.temperatura = valor;
    this.notificar();
  }

  private notificar() {
    for (const observer of this.observers) {
      observer.update(this.temperatura);
    }
  }
}
```

Agora criamos observadores concretos.

```
class DisplayTemperatura implements Observer {
  update(temperatura: number) {
    console.log("Display atualizado:", temperatura);
  }
}

class AlarmeTemperatura implements Observer {
  update(temperatura: number) {
    if (temperatura > 30) {
      console.log("Alerta: temperatura muito alta!");
    }
  }
}
```

Utilizando o sistema:

```
const sensor = new SensorTemperatura();

const display = new DisplayTemperatura();
const alarme = new AlarmeTemperatura();

sensor.adicionarObserver(display);
sensor.adicionarObserver(alarme);

sensor.setTemperatura(25);
sensor.setTemperatura(35);
```

Nesse exemplo, sempre que a temperatura é atualizada, todos os observadores registrados são notificados automaticamente.

2.4. Benefícios do Observer Pattern

O uso do Observer Pattern traz diversas vantagens para o design de software.

Entre os principais benefícios estão:

- baixo acoplamento entre componentes
- facilidade para adicionar novos observadores
- organização da comunicação entre objetos
- maior flexibilidade do sistema

Esse padrão permite que diferentes partes do sistema reajam a eventos sem depender diretamente umas das outras.

2.6. Onde o Observer é utilizado

O Observer Pattern aparece com frequência em frameworks modernos de interface, especialmente em bibliotecas baseadas em **componentes reativos**, como o React.

No React, a interface do usuário é atualizada automaticamente quando ocorre uma **mudança de estado (state)**.

Nesse contexto, os componentes da interface podem ser vistos como **observadores** que reagem às alterações de dados.

Quando o estado de um componente é modificado, o React executa um novo processo de renderização e atualiza automaticamente todos os componentes que dependem desse estado.

Considere o exemplo simplificado a seguir:

```
import { useState } from "react";

export default function Contador() {
  const [contador, setContador] = useState(0);

  return (
    <div>
      <p>Valor: {contador}</p>
      <button onClick={() => setContador(contador + 1)}>Incrementar</button>
    </div>
  );
}
```

Nesse exemplo:

- a variável `contador` representa o estado observado
- o componente da interface **observa esse estado**
- quando `setContador` é chamado, o estado é atualizado

- o React **notifica automaticamente o componente**, que é re-renderizado

Embora o React não implemente explicitamente o padrão Observer da mesma forma que os exemplos clássicos da orientação a objetos, o comportamento do framework segue a **mesma ideia fundamental**: componentes reagem automaticamente a mudanças de estado.

Esse modelo reativo é amplamente utilizado em:

- bibliotecas de gerenciamento de estado (Redux, Zustand, MobX)
- sistemas de eventos
- interfaces baseadas em componentes
- aplicações web modernas

Compreender o Observer Pattern ajuda o desenvolvedor a entender melhor como frameworks como o React organizam a atualização da interface e a comunicação entre componentes.

iii. Strategy Pattern

3.1. Introdução

Em muitos sistemas de software, é comum que um mesmo problema possa ser resolvido utilizando **diferentes algoritmos ou estratégias**.

Considere, por exemplo, um sistema de e-commerce que precisa calcular o valor do frete de um pedido. Dependendo da transportadora escolhida, o cálculo pode variar.

Alguns exemplos de estratégias possíveis seriam:

- cálculo de frete via Correios
- cálculo via transportadora privada
- cálculo com frete fixo
- cálculo com frete grátis acima de determinado valor

Uma implementação simples poderia utilizar várias estruturas condicionais, como if ou switch, para escolher qual algoritmo executar.

Entretanto, à medida que novas estratégias são adicionadas, esse tipo de solução pode tornar o código:

- difícil de manter
- difícil de expandir
- fortemente acoplado

Para resolver esse tipo de problema, podemos utilizar o Strategy Pattern.

3.2. O que é o Strategy Pattern

O Strategy Pattern é um padrão comportamental que permite **definir uma família de algoritmos**, encapsular cada um deles em uma classe e torná-los intercambiáveis.

Em outras palavras, o padrão permite que o comportamento de um objeto seja alterado **sem modificar o código da classe principal**.

Cada estratégia representa uma forma diferente de realizar uma determinada operação.

O objeto principal apenas utiliza a estratégia selecionada, sem precisar conhecer os detalhes de sua implementação.

Isso torna o sistema mais flexível e facilita a adição de novos comportamentos.

3.3. Exemplo em TypeScript

Considere um sistema de cálculo de desconto em uma loja online.

Primeiro definimos a interface da estratégia.

```
interface DescontoStrategy {
  calcular(valor: number): number;
}
```

Agora criamos algumas estratégias concretas.

```
class DescontoPadrao implements DescontoStrategy {
  calcular(valor: number): number {
    return valor * 0.9;
  }
}

class DescontoVip implements DescontoStrategy {
  calcular(valor: number): number {
    return valor * 0.8;
  }
}

class SemDesconto implements DescontoStrategy {
  calcular(valor: number): number {
    return valor;
  }
}
```

Agora criamos a classe que utiliza a estratégia.

```
class Carrinho {
  constructor(private estrategia: DescontoStrategy) {}

  calcularTotal(valor: number): number {
    return this.estrategia.calcular(valor);
  }
}
```

Utilizando o sistema:

```
const estrategia = new DescontoVip();
```

```
const carrinho = new Carrinho(estrategia);
const total = carrinho.calcularTotal(100);
console.log("Total com desconto:", total);
```

Nesse exemplo, o carrinho utiliza uma estratégia específica para calcular o desconto, mas outras estratégias podem ser utilizadas sem modificar a classe principal.

3.4. Benefícios do Strategy Pattern

O uso do Strategy Pattern traz diversas vantagens para o desenvolvimento de software.

Entre os principais benefícios estão:

- possibilidade de trocar algoritmos dinamicamente
- redução do uso de estruturas condicionais complexas
- maior organização do código
- facilidade para adicionar novos comportamentos

Além disso, o padrão ajuda a manter o código alinhado ao **princípio aberto/fechado (Open/Closed Principle)**, permitindo que novos comportamentos sejam adicionados sem alterar o código existente.

iv. Command Pattern

4.1. Introdução

Em muitos sistemas de software, é comum que determinadas ações precisem ser executadas a partir de solicitações feitas por diferentes partes da aplicação.

Por exemplo, em uma interface gráfica, um botão pode executar diversas operações, como:

- salvar um documento
- enviar um formulário
- excluir um registro
- gerar um relatório

Uma implementação simples poderia fazer com que o botão chamasse diretamente os métodos responsáveis por essas operações.

Entretanto, quando a interface depende diretamente das classes que executam as ações, o sistema pode se tornar **fortemente acoplado**. Isso dificulta a reutilização dos componentes e torna o código mais difícil de manter.

Uma forma de resolver esse problema é encapsular cada ação em um objeto separado.

Essa é a ideia central do Command Pattern.

4.2. O que é o Command Pattern

O Command Pattern é um padrão comportamental que transforma uma **requisição ou ação em um objeto**.

Esse objeto contém todas as informações necessárias para executar a operação.

Ao encapsular comandos em objetos, o sistema passa a tratar ações da mesma forma que trata outros objetos do programa. Isso permite, por exemplo:

- armazenar comandos em listas ou filas
- executar comandos posteriormente
- desfazer ou refazer operações
- desacoplar quem solicita a ação de quem a executa

Dessa forma, o objeto que solicita a operação não precisa conhecer os detalhes de sua implementação.

4.3. Exemplo em TypeScript

Considere um sistema simples de controle de iluminação.

Primeiro definimos a interface do comando.

```
interface Command {
  executar(): void;
}
```

Agora criamos o objeto que realmente executa a ação.

```
class Luz {
  ligar() {
    console.log("Luz ligada");
  }

  desligar() {
    console.log("Luz desligada");
  }
}
```

Agora criamos comandos concretos.

```
class LigarLuzCommand implements Command {
  constructor(private luz: Luz) {}

  executar() {
    this.luz.ligar();
  }
}

class DesligarLuzCommand implements Command {
  constructor(private luz: Luz) {}

  executar() {
    this.luz.desligar();
  }
}
```

Agora criamos o invoker.

```
class ControleRemoto {
  constructor(private comando: Command) {}

  pressionarBotao() {
    this.comando.executar();
  }
}
```

Utilizando o sistema:

```
const luz = new Luz();

const ligar = new LigarLuzCommand(luz);
const desligar = new DesligarLuzCommand(luz);

const controle = new ControleRemoto(ligar);
controle.pressionarBotao();

const controle2 = new ControleRemoto(desligar);
controle2.pressionarBotao();
```

Nesse exemplo, o **controle remoto** não conhece a implementação da operação. Ele apenas executa um comando.

4.4. Benefícios do Command Pattern

O uso do Command Pattern traz diversas vantagens para o design de software.

Entre os principais benefícios estão:

- desacoplamento entre quem solicita a ação e quem a executa
- possibilidade de armazenar comandos em filas ou listas
- facilidade para implementar funcionalidades de **desfazer e refazer (undo/redo)**
- maior flexibilidade na organização das ações do sistema

Além disso, o padrão permite tratar comandos como **objetos independentes**, facilitando sua reutilização e composição.

v. State Pattern

5.1. Introdução

Em muitos sistemas de software, o comportamento de um objeto depende do **estado em que ele se encontra**.

Considere, por exemplo, um sistema que representa um pedido em um e-commerce. Um pedido pode passar por diferentes estados durante seu ciclo de vida, como:

- aguardando pagamento
- pago
- enviado
- entregue

Dependendo do estado atual do pedido, determinadas ações podem ou não ser permitidas.

Por exemplo:

- um pedido **aguardando pagamento** pode ser cancelado
- um pedido **enviado** não pode mais ser cancelado
- um pedido **entregue** não pode ser modificado

Uma implementação simples poderia utilizar várias estruturas condicionais (if ou switch) para verificar o estado atual e decidir qual comportamento executar.

Entretanto, à medida que o número de estados cresce, esse tipo de solução pode tornar o código:

- difícil de manter
- difícil de entender
- propenso a erros

Para organizar melhor esse tipo de lógica, podemos utilizar o State Pattern.

5.2. O que é o State Pattern

O State Pattern é um padrão comportamental que permite que um objeto altere seu comportamento quando seu **estado interno muda**.

Nesse padrão, cada estado é representado por uma **classe específica** que implementa um comportamento próprio.

Em vez de utilizar várias estruturas condicionais para verificar o estado atual, o objeto delega o comportamento para o objeto que representa seu estado.

Isso permite que o sistema trate cada estado como uma entidade independente, tornando o código mais organizado e flexível.

5.3. Exemplo em TypeScript

Considere um sistema simples de pedidos.

Primeiro definimos a interface de estado.

```
interface EstadoPedido {
  processar(pedido: Pedido): void;
}
```

Agora criamos alguns estados concretos.

```
class PedidoAguardandoPagamento implements EstadoPedido {
  processar(pedido: Pedido): void {
    console.log("Pagamento confirmado.");
    pedido.setEstado(new PedidoPago());
  }
}

class PedidoPago implements EstadoPedido {
  processar(pedido: Pedido): void {
```

```
      console.log("Pedido enviado para entrega.");
      pedido.setEstado(new PedidoEnviado());
    }
  }

  class PedidoEnviado implements EstadoPedido {
    processar(pedido: Pedido): void {
      console.log("Pedido entregue ao cliente.");
    }
  }
```

Agora criamos a classe principal.

```
class Pedido {
  private estado: EstadoPedido;

  constructor() {
    this.estado = new PedidoAguardandoPagamento();
  }

  setEstado(estado: EstadoPedido) {
    this.estado = estado;
  }

  processarPedido() {
    this.estado.processar(this);
  }
}
```

Utilizando o sistema:

```
const pedido = new Pedido();

pedido.processarPedido();
pedido.processarPedido();
pedido.processarPedido();
```

Nesse exemplo, o comportamento do objeto **Pedido** muda automaticamente conforme seu estado evolui.

5.4. Benefícios do State Pattern

O uso do State Pattern traz diversas vantagens para o design de software.

Entre os principais benefícios estão:

- eliminação de estruturas condicionais complexas
- melhor organização da lógica relacionada aos estados
- facilidade para adicionar novos estados
- maior clareza no comportamento do sistema

Cada estado passa a possuir **responsabilidade própria**, tornando o código mais modular e fácil de manter.

5.5. Onde o State é utilizado

O State Pattern aparece frequentemente em sistemas que possuem **fluxos de estados bem definidos**.

Alguns exemplos incluem:

- sistemas de pedidos em e-commerce
- workflows de aprovação
- controle de estados em jogos
- protocolos de comunicação
- máquinas de estado em sistemas embarcados

Esse padrão também é utilizado em aplicações que precisam representar **diferentes fases de um processo**, permitindo que o comportamento do sistema mude de forma organizada conforme o estado evolui.

vi. Chain of Responsability Pattern

6.1. Introdução

Em muitos sistemas de software, uma solicitação pode ser tratada por **diferentes objetos**, dependendo de determinadas condições.

Considere, por exemplo, um sistema de aprovação de despesas em uma empresa. Dependendo do valor da solicitação, diferentes níveis hierárquicos podem ser responsáveis pela aprovação:

- despesas até **R$ 500** podem ser aprovadas por um supervisor
- despesas até **R$ 2000** podem ser aprovadas por um gerente
- valores maiores podem exigir aprovação de um diretor

Uma implementação simples poderia utilizar várias estruturas condicionais (if ou switch) para verificar o valor e decidir quem deve aprovar a solicitação.

Entretanto, esse tipo de abordagem pode gerar:

- forte acoplamento entre os componentes
- código difícil de modificar
- dificuldade para adicionar novos níveis de aprovação

Uma solução mais flexível consiste em organizar os responsáveis em uma **cadeia de processamento**.

Essa é a ideia central do Chain of Responsibility Pattern.

6.2. O que é o Chain of Responsibility

O Chain of Responsibility Pattern é um padrão comportamental que permite que uma requisição seja passada por uma **cadeia de objetos**, onde cada objeto decide se pode tratá-la ou se deve repassá-la para o próximo elemento da cadeia.

Nesse modelo:

- cada objeto possui uma referência para o **próximo elemento da cadeia**
- cada elemento decide se consegue tratar a requisição
- caso não consiga, a requisição é encaminhada para o próximo objeto

Dessa forma, o objeto que faz a solicitação **não precisa saber qual elemento da cadeia irá processá-la**.

6.3. Estrutura do Chain of Responsibility

De forma geral, o padrão envolve os seguintes elementos:

Elemento	Função
Handler	interface que define o método de tratamento da requisição
Concrete Handler	implementação concreta que processa ou repassa a requisição
Client	cria e inicia a requisição

O funcionamento ocorre da seguinte forma:

1. O cliente envia uma requisição para o primeiro elemento da cadeia.
2. O elemento verifica se pode tratar a requisição.
3. Caso não possa, ele encaminha a requisição para o próximo elemento.
4. O processo continua até que algum elemento trate a requisição.

6.4. Exemplo em TypeScript

Considere um sistema de aprovação de despesas.

Primeiro definimos a classe base para os aprovadores.

```
abstract class Aprovador {
  protected proximo?: Aprovador;

  setProximo(aprovador: Aprovador) {
    this.proximo = aprovador;
  }

  abstract aprovar(valor: number): void;
}
```

Agora criamos as implementações concretas.

```
class Supervisor extends Aprovador {
  aprovar(valor: number): void {
    if (valor <= 500) {
      console.log("Supervisor aprovou a despesa:", valor);
    } else if (this.proximo) {
      this.proximo.aprovar(valor);
    }
  }
}
```

```
class Gerente extends Aprovador {
  aprovar(valor: number): void {
    if (valor <= 2000) {
      console.log("Gerente aprovou a despesa:", valor);
    } else if (this.proximo) {
      this.proximo.aprovar(valor);
    }
  }
}

class Diretor extends Aprovador {
  aprovar(valor: number): void {
    console.log("Diretor aprovou a despesa:", valor);
  }
}
```

Agora configuramos a cadeia de responsabilidade.

```
const supervisor = new Supervisor();
const gerente = new Gerente();
const diretor = new Diretor();

supervisor.setProximo(gerente);
gerente.setProximo(diretor);

supervisor.aprovar(300);
supervisor.aprovar(1500);
supervisor.aprovar(5000);
```

Nesse exemplo, cada solicitação percorre a cadeia até encontrar um responsável capaz de tratá-la.

6.5. Benefícios do Chain of Responsibility

O uso do Chain of Responsibility Pattern traz diversas vantagens para o design de software.

Entre os principais benefícios estão:

- redução do acoplamento entre o solicitante e os processadores
- possibilidade de reorganizar facilmente a cadeia de processamento
- maior flexibilidade para adicionar novos manipuladores
- melhor organização da lógica de tratamento de requisições

Esse padrão permite que diferentes objetos participem do processamento de uma solicitação **sem que o cliente precise conhecer todos eles**.

6.6. Onde o Chain of Responsibility é utilizado

O Chain of Responsibility aparece com frequência em sistemas que possuem **múltiplos níveis de processamento de requisições**.

Alguns exemplos incluem:

- sistemas de **aprovação hierárquica**
- processamento de **requisições HTTP em servidores web**
- **middlewares em frameworks backend**
- validação de dados em etapas
- sistemas de tratamento de eventos

Um exemplo bastante comum ocorre em frameworks backend como Express.js, onde as requisições passam por uma sequência de **middlewares**, e cada um deles pode processar ou encaminhar a requisição para o próximo elemento da cadeia.

Exercícios

Veja os vídeos se tiver dúvidas:

Exercício 1: https://youtu.be/_LH1oM1knjY

Exercício 2: https://youtu.be/MAtzAu34ZgY

Exercício 3: https://youtu.be/Xv1wWkZ6cfo

Exercício 4: https://youtu.be/qIErNdBVIxw

Exercício 5: https://youtu.be/po39YrQgeZw

Exercício 1 – Aplicando o Observer Pattern

Contexto

Considere um sistema de monitoramento de estoque em uma loja online.

Sempre que a quantidade de um produto for atualizada, diferentes partes do sistema precisam ser notificadas:

- um sistema de **notificação** para avisar administradores
- um sistema de **interface** que atualiza a tela de estoque

Para evitar que o sistema de estoque dependa diretamente desses componentes, será utilizado o Observer Pattern.

Parte A – Criando a interface Observer

Crie uma interface chamada `Observer` contendo o método:

```
update(quantidade: number): void
```

Parte B – Criando o Subject

Crie uma classe chamada `Estoque` contendo:

- um atributo para armazenar a lista de observadores
- métodos:

```
adicionarObserver(observer)
removerObserver(observer)
atualizarEstoque(valor)
```

Sempre que o estoque for atualizado, todos os observadores devem ser notificados.

Parte C – Criando observadores concretos

Crie duas classes:

- `NotificacaoEstoque`
- `DisplayEstoque`

Cada uma deve implementar o método `update`.

Exercício 2 – Aplicando o Strategy Pattern

Contexto

Considere um sistema de cálculo de frete para uma loja virtual.

O valor do frete pode ser calculado utilizando diferentes estratégias:

- frete econômico
- frete expresso

O objetivo é aplicar o Strategy Pattern para permitir que o algoritmo de cálculo seja facilmente substituído.

Parte A – Criando a interface

Crie uma interface chamada `FreteStrategy` contendo:

```
calcular(valor: number): number
```

Parte B – Criando estratégias

Crie duas classes:

- `FreteEconomico`
- `FreteExpresso`

Parte C – Criando o contexto

Crie uma classe chamada `Pedido` que recebe a estratégia de frete.

Exercício 3 – Aplicando o Command Pattern

Contexto

Considere um sistema de controle de ventilador inteligente.

O ventilador pode:

- ligar
- desligar

Utilize o Command Pattern para encapsular essas ações.

Parte A – Criando a interface Command

```
executar(): void
```

Parte B – Criando o Receiver

Crie a classe `Ventilador` com métodos:

```
ligar()
desligar()
```

Parte C – Criando os comandos

Crie:

- `LigarVentiladorCommand`
- `DesligarVentiladorCommand`

Exercício 4 – Aplicando o State Pattern

Contexto

Considere um sistema que representa um semáforo.

O semáforo pode possuir três estados:

- verde
- amarelo
- vermelho

Cada estado deve definir o próximo estado.

Parte A – Criando a interface

```
mudar(semaforo): void
```

Parte B – Criando estados concretos

Crie:

- Verde
- Amarelo
- Vermelho

Parte C – Criando o contexto

Crie a classe `Semaforo`.

Exercício 5 – Aplicando o Chain of Responsibility

Contexto

Considere um sistema de aprovação de compras.

- até 100 → aprovado pelo supervisor
- até 1000 → aprovado pelo gerente
- acima disso → aprovado pelo diretor

Parte A – Criando a classe base

Crie uma classe abstrata chamada `Aprovador`.

Parte B – Criando os aprovadores

Crie:

- Supervisor
- Gerente
- Diretor

Parte C – Criando a cadeia

Conecte os aprovadores.